

Sistemas Operativos

1er Recuperatorio 1º Parcial 1C2025 – TM

Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

- 1) a) Se puede justificar por ambos casos según el contexto:
 - Falso. Cuando ocurre una interrupción (por ejemplo, de hardware), se interrumpe la ejecución (como última etapa del ciclo de instrucción actual) y el SO atiende la misma antes de reanudar la ejecución de la syscall. La atención de la interrupción es prioritaria, se guarda el contexto de la syscall en curso para poder reanudarla luego.
 - Verdadero. Si las interrupciones se encuentran deshabilitadas como parte de la ejecución de la syscall (por ejemplo, para garantizar mutua exclusión) se esperará a que se vuelvan a habilitar y finalice la ejecución para atender la interrupción.
- b) Falso. Las interrupciones sincrónicas (también conocidas como excepciones) son causadas por eventos que ocurren como resultado directo de la ejecución de una instrucción, como divisiones por cero o accesos inválidos a memoria. Estas no son generadas necesariamente por el sistema operativo como tal, sino que son generadas por el propio hardware o por la lógica del procesador en respuesta a un error en la ejecución, para luego ser atendidas por el SO.
- 2) a) Falso. Si bien un hilo no debería acceder al stack de otro hilo, la protección del acceso a direcciones de memoria por parte del sistema operativo es a nivel PROCESO. Por este motivo, no tenemos protecciones a nivel SO que eviten que mediante una manipulación maliciosa de las direcciones de memoria se pueda acceder desde un KLT al stack de otro KLT del mismo proceso.
- b) Falso. Por ser ULTs, el sistema operativo no reconoce a los hilos, por lo que no pueden ser ejecutados en paralelo aunque se utilice jacketing. Podría ocurrir solamente si ese proceso a su vez tiene KLTs.

3) Un ejemplo de implementación:

- Una cola por cada nivel de prioridad.
- Los procesos nuevos se agregan a la cola correspondiente a su prioridad.
- Cada cola implementa un RR para garantizar previsibilidad en el tiempo de espera.
- Se implementa aging para evitar starvation, subiendo la prioridad (y por ende cambiando de cola) a aquellos procesos que esperan en ready durante X tiempo.
- Sin desalojo entre colas.

Podría incluirse bajar la prioridad a aquellos procesos que utilizan todo su quantum (salvo que ya estén en la prioridad más baja).

4) Condiciones de Bernstein: más de un proceso/hilo accediendo de forma concurrente a un mismo recurso, estando al menos uno de ellos modificándolo. Un proceso de usuario no podría solucionar sus condiciones de carrera deshabilitando interrupciones ya que éste es un mecanismo que sólo puede ser utilizado por el sistema operativo (instrucción privilegiada).

5)

	¿Cuándo se aplica?	Flexibilidad de peticiones	¿Puede ocurrir deadlock?	Overhead
Detección	Deadlock	Muy flexible, cualquier petición válida puede realizarse.	Si	Depende de la frecuencia del algoritmo de detección.
Evasión	Asignación	Intermedia, los procesos deben declarar sus peticiones máximas.	No	Alto, se debe correr el algoritmo del banquero con cada petición.

Ejemplo de aplicación:

“Sistema usado en un puesto administrativo de una empresa, donde el operador usa planillas de cálculo, imprime documentos y navega por internet.”

La criticidad del deadlock en este sistema es baja, por lo que la utilización de detección y recuperación es posible y tiene las siguientes ventajas:

- Menos overhead: no debe correrse el algoritmo del banquero.
- Mayor flexibilidad en la petición de recursos → no deben declararse recursos máximos a utilizar por proceso.

Práctica

1) a)

PA																			
PB																			
PC																			
PD																			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	

b)

Un Alfa igual a 0 significaría que el estimador mantuviera siempre su estimación inicial ya que no le daría ningún “peso” a lo que realmente ejecutó para re-estimar. Un alfa igual a 1 re-estimaría siempre tomando la última ráfaga que ejecutó.

El diagrama cambiaría en el instante t=12, ya que la estimación de la ráfaga de PC sería 3, igual a lo que le queda de estimación a PA, desempatao por FIFO seguiría PA.

2)

pedidoListo[N] = {0, ..., 0}

atendido = 0

mutexPedidos = 1

clienteEnCola = 0

pedidoRealizado = 0

pedidosPendientes = 0

contadorSabores[3] = [0,0,0]

Cliente (N instancias)	Cajero (1 instancia)	Heladero (1 instancia)	Fábrica (3 instancias)
posicionarse() signal(clienteEnCola) wait(atendido) hacerPedido() signal(pedidoRealizado) wait(pedidoListo[id()]) retirarHelado()	while(true) { wait(clienteEnCola) llamarCliente() signal(atendido) wait(pedidoRealizado) pedido = tomarPedido() wait(mutexPedidos) agregar(pedidos, pedido) signal(mutexPedidos) signal(pedidosPendientes) } 	while(true) { wait(pedidosPendientes) wait(mutexPedidos) pedido = retirar(pedidos) signal(mutexPedidos) idSabor = pedido->sabor wait(contadorSabores[idSabor]) entregarHelado() idCliente = pedido->cliente signal(pedidoListo[idCliente]) } 	while(true) { entregarHelado() signal(contadorSabores[0]) x50 signal(contadorSabores[1]) x50 signal(contadorSabores[2]) x50 esperarDosHoras() }

3) a) PC **wait**(valorLeido), **wait**(mutexAcum), printf(), **signal**(mutexAcum), **wait**(valorLeido)

PA **wait**(mutexAcum), **wait**(valorGenerado) => bloqueado (valorGenerado=-1, espera por PB)

PB **wait**(valorLeido) => bloqueado (valorLeido=-1, espera por PA)

PC **wait**(mutexAcum) => bloqueado (mutexAcum=-1, espera por PA)

b) PB **wait**(valorLeido), ultValor = gv(), **signal**(valorGenerado), **wait**(valorLeido)

PA **wait**(mutexAcum), **wait**(valorGenerado), acum += ultValor

En este momento la próxima instrucción de PB modifica la variable compartida “ultValor” al mismo tiempo que PA la lee y su próxima instrucción la modifica, pudiendo generar una condición de carrera.

En ambos casos existen distintos órdenes de ejecución que cumplen con las condiciones pedidas.